

TP2A - RestAPI - Servidores sin control de estado y Gestión de la Base de Datos

Nombre: Julieta Barahona Roth, Mia Bertola

Consigna del TP 2A. Desarrollo de Cliente - Servidor bajo Arquitectura Rest

1. Analizar la aplicación Flask básica. Explicar el funcionamiento de `app = Flask(name)`, `@app.route("/")`, `jsonify()` y el modo `debug=True`, tomando como base `Prueba_Flask_Routes_01.py`.
2. Implementar una API REST de sensores.
Desarrollar endpoints para listar sensores, consultar un sensor por id, crear un sensor, modificarlo y eliminarlo usando métodos GET, POST, PUT y DELETE.
3. Corregir y mejorar el manejo de IDs
Revisar el acceso directo a listas mediante `sensors[id]` y reemplazarlo por una búsqueda segura por campo id, devolviendo error 404 si no existe.
4. Comparar GET vs POST en Flask
Implementar dos rutas de login: una con parámetros por URL y otra con formulario POST. Explicar las diferencias de visibilidad, seguridad básica y uso correcto de `request.args` y `request.form`.
5. Persistir lecturas de sensores en SQLite
Crear una tabla `lectura_sensores` con campos como `co2`, `temp`, `hum`, `fecha`, `lugar`, `altura`, `presion`, `presion_nm` y `temp_ext`, siguiendo la estructura usada en `sensores_rx.py`.
6. Simular capturas de sensores ambientales
Generar lecturas aleatorias de CO₂, temperatura y humedad; almacenarlas en la base de datos y permitir configurar cantidad de capturas e intervalo entre mediciones.
7. Integrar datos externos de clima
Usar una función similar a `geo_latlon()` para obtener temperatura exterior, presión y humedad desde una API climática, y relacionar esos datos con las lecturas internas.
8. Explicar las razones por la cual lo anterior es una Arquitectura Cliente Servidor Rest. Indicar si se cumplen los requisitos. Explicar dónde se definen el cliente y el servidor. Qué elementos faltan.
9. Comparar este modelo con el desarrollado en el TP1.
10. Desarrollar cliente visual WebSocket
Crear una interfaz web que se conecte a un servidor WebSocket, muestre estado de conexión, permite enviar mensajes y visualice respuestas en tiempo real, según el notebook `Cliente_Servidor_Websockets_R2.ipynb`.

1. `prueba_flask_routes_01.py`

`app = Flask(name)` → crea la aplicación Flask y a partir de este objeto se inicializa el servidor web y prepara las rutas HTTP. El parámetro `__name__` permite a Flask identificar el módulo principal de la aplicación y localizar correctamente recursos asociados como templates.

`@app.route("/")` → define la ruta, osea que si un cliente ejecuta la / específica se ejecuta la función predeterminada para esa /

`jsonify()` → convierte listas y diccionarios de Python a JSON, donde JSON es el formato más utilizado en APIs REST para intercambiar información entre el cliente y servidor

`debug = True` → muestra errores y reinicia automático Flask, facilita el desarrollo y las pruebas de la aplicación.

2. Implementar una API REST de sensores.

Desarrollar endpoints para listar sensores, consultar un sensor por id, crear un sensor, modificarlo y eliminarlo usando métodos GET, POST, PUT y DELETE.

Adjunto código comentado y detallado

```
127.0.0.1
[1] 127.0.0.1

{
  "sensor": {
    "co2": 950,
    "fecha": "08/05/2026",
    "hum": 58.0,
    "id": 2,
    "temp": 26.0
  }
}

julietabarahonaroth -- zsh -- 79x26
-H "Content-Type: application/json" \
-d '{"co2":950,"temp":25.5,"hum":60.2,"fecha":"08/05/2026"}'
{
  "mensaje": "Sensor creado correctamente",
  "sensor": {
    "co2": 950,
    "fecha": "08/05/2026",
    "hum": 60.2,
    "id": 2,
    "temp": 25.5
  }
}
julietabarahonaroth@192 ~ % curl -X PUT http://127.0.0.1:5001/sensors/2 \
-H "Content-Type: application/json" \
-d '{"temp":26.0,"hum":58.0}'
{
  "mensaje": "Sensor actualizado correctamente",
  "sensor": {
    "co2": 950,
    "fecha": "08/05/2026",
    "hum": 58.0,
    "id": 2,
    "temp": 26.0
  }
}

tp2A -- Python -- Python prueba_flask_route_2.py -- 80x16
ce&f=console.png HTTP/1.1" 304 -
127.0.0.1 - - [08/May/2026 23:43:54] "GET /sensors/2?__debugger__=yes&cmd=resource&f=console.png&s=GQGMfTE5CVx8mo3J5BIW HTTP/1.1" 304 -
127.0.0.1 - - [08/May/2026 23:43:54] "GET /sensors/1 HTTP/1.1" 200 -
127.0.0.1 - - [08/May/2026 23:46:04] "POST /sensors HTTP/1.1" 201 -
127.0.0.1 - - [08/May/2026 23:46:11] "PUT /sensors/2 HTTP/1.1" 200 -
127.0.0.1 - - [08/May/2026 23:46:19] "GET /sensors/2 HTTP/1.1" 200 -
127.0.0.1 - - [08/May/2026 23:46:19] "GET /sensors/2?__debugger__=yes&cmd=resource&f=debugger.js HTTP/1.1" 304 -
127.0.0.1 - - [08/May/2026 23:46:19] "GET /sensors/2?__debugger__=yes&cmd=resource&f=console.png HTTP/1.1" 304 -
127.0.0.1 - - [08/May/2026 23:46:20] "GET /sensors/2?__debugger__=yes&cmd=resource&f=console.png&s=GQGMfTE5CVx8mo3J5BIW HTTP/1.1" 304 -
127.0.0.1 - - [08/May/2026 23:46:20] "GET /sensors/2?__debugger__=yes&cmd=resource&f=style.css HTTP/1.1" 304 -
```

The screenshot shows a web browser at 127.0.0.1 displaying a JSON error response: `{ "error": "Sensor no encontrado" }`. Below the browser, a terminal window titled 'julietabarahonaroth — zsh — 79x26' shows the following commands and outputs:

```
julietabarahonaroth@192 ~ % curl -X PUT http://127.0.0.1:5001/sensors/2 \
-H "Content-Type: application/json" \
-d '{"temp":26.0,"hum":58.0}'
{"mensaje": "Sensor actualizado correctamente",
 "sensor": {
  "co2": 950,
  "fecha": "08/05/2026",
  "hum": 58.0,
  "id": 2,
  "temp": 26.0
 }}
julietabarahonaroth@192 ~ %
julietabarahonaroth@192 ~ %
julietabarahonaroth@192 ~ % curl -X DELETE http://127.0.0.1:5001/sensors/2
{"mensaje": "Sensor eliminado correctamente",
 "result": true
}
julietabarahonaroth@192 ~ %
```

Below the terminal, a Python script window titled 'tp2A — Python · Python prueba_flask_route_2.py — 80x16' shows a log of HTTP requests and responses:

```
ce&f=console.png&s=GQGMfTE5CVx8mo3J5BIW HTTP/1.1" 304 -
127.0.0.1 - - [08/May/2026 23:43:54] "GET /sensors/1 HTTP/1.1" 200 -
127.0.0.1 - - [08/May/2026 23:46:04] "POST /sensors HTTP/1.1" 201 -
127.0.0.1 - - [08/May/2026 23:46:11] "PUT /sensors/2 HTTP/1.1" 200 -
127.0.0.1 - - [08/May/2026 23:46:19] "GET /sensors/2 HTTP/1.1" 200 -
127.0.0.1 - - [08/May/2026 23:46:19] "GET /sensors/2?__debugger__=yes&cmd=resource&f=debugger.js HTTP/1.1" 304 -
127.0.0.1 - - [08/May/2026 23:46:19] "GET /sensors/2?__debugger__=yes&cmd=resource&f=console.png HTTP/1.1" 304 -
127.0.0.1 - - [08/May/2026 23:46:20] "GET /sensors/2?__debugger__=yes&cmd=resource&f=console.png&s=GQGMfTE5CVx8mo3J5BIW HTTP/1.1" 304 -
127.0.0.1 - - [08/May/2026 23:46:20] "GET /sensors/2?__debugger__=yes&cmd=resource&f=style.css HTTP/1.1" 304 -
127.0.0.1 - - [08/May/2026 23:47:01] "DELETE /sensors/2 HTTP/1.1" 200 -
127.0.0.1 - - [08/May/2026 23:47:05] "GET /sensors/2 HTTP/1.1" 404 -
```

3. Corregir y mejorar el manejo de IDs

Revisar el acceso directo a listas mediante `sensors[id]` y reemplazarlo por una búsqueda segura por campo id, devolviendo error 404 si no existe.

anteriormente hacíamos → `return jsonify({'Sensors':sensors[id]})`

El programa asume que el ID y la posición de la lista es lo mismo, y no funciona porque si yo elimino un sensor, la lista cambia de posiciones y ya no queda el sensor ID 1 con la posición en la lista 1, y también puede romper el código.

código mejorado: recorre todos los sensores, si lo encuentra al ID lo devuelve sino tira un mensaje.

for sensor in sensors:

if sensor["id"] == id:

return jsonify({"sensor": sensor})

- /sensors/15 → donde 15 es un IDENTIFICADOR LÓGICO no una posición en la lista

4. Comparar GET vs POST en Flask

Implementar dos rutas de login: una con parámetros por URL y otra con formulario POST. Explicar las diferencias de visibilidad, seguridad básica y uso correcto de request.args y request.form.

En Flask, una solicitud GET se utiliza para consultar información. Los datos se envían como parámetros dentro de la URL, después del signo **?**. Por eso, en la ruta **/login_get**, el nombre y la clave se leen con **request.args.get()**, ya que **request.args** contiene los parámetros enviados por URL. Los datos son visibles en la barra del navegador y esto no es recomendable para contraseñas, porque pueden quedar guardadas en el historial, en favoritos, en capturas de pantalla, etc.

Una solicitud POST se utiliza para enviar datos al servidor dentro del cuerpo de la petición HTTP. En la ruta **/login**, los datos del formulario se leen con **request.form.get()**, porque vienen desde un formulario HTML enviado con **method="post"** donde el URL queda solo **/login** y los valores ingresados no aparecen en la barra del navegador.

El POST es más adecuado para formularios de login o envío de datos sensibles, lo que si es que no garantiza seguridad completa por sí solo pero por lo menos no muestra datos en la barra del navegador. Para proteger realmente los datos se debería usar HTTPS.

5. Persistir lecturas de sensores en SQLite

Crear una tabla **lectura_sensores** con campos como **co2**, **temp**, **hum**, **fecha**, **lugar**, **altura**, **presion**, **presion_nm** y **temp_ext**, siguiendo la estructura usada en **sensores_rx.py**.

Se define una tabla de lectura de sensores → **class LecturaSensores(db.Model)**

Donde la tabla contiene: **co2**, **temp**, **hum**, **fecha**, **lugar**, **altura**, **presion**, **presion_nm**, **temp_ext**

Cada atributo definido con **db.Column(...)** que representa una columna SQL dentro de SQLite.

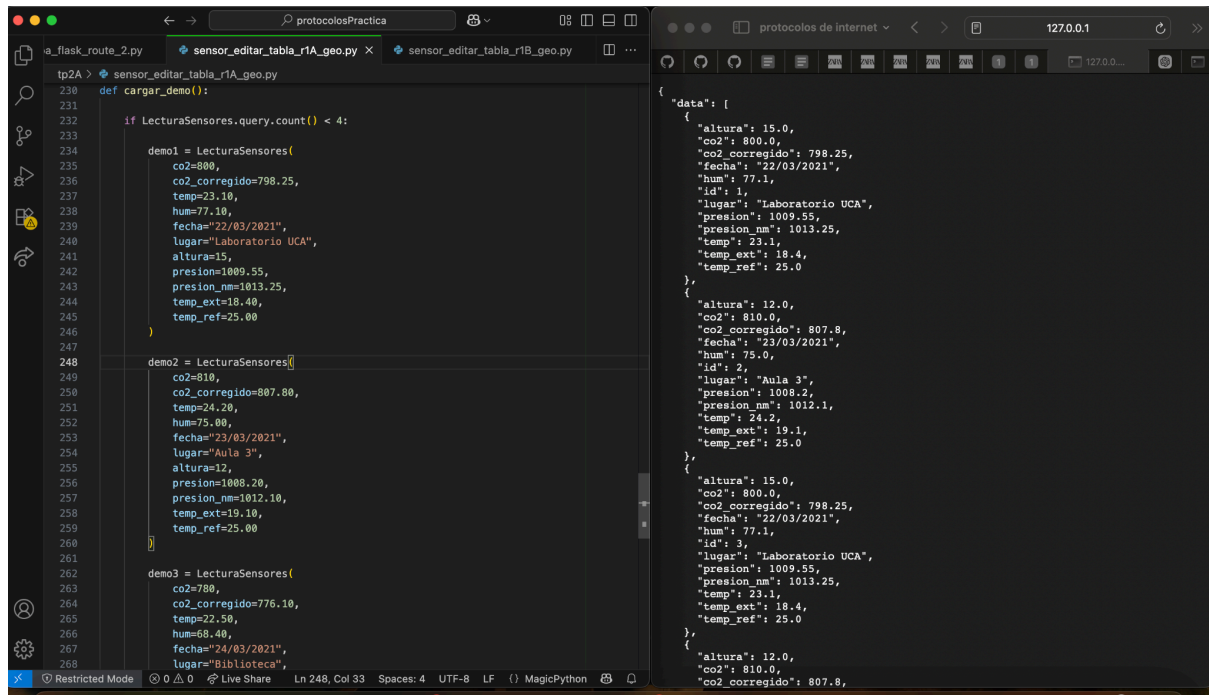
La base de datos se configura con:

- **app.config["SQLALCHEMY_DATABASE_URI"] = "sqlite://datos_sensores.db"**

y las tablas se crean automaticamente con `→ db.create_all()`.

`/cargar_demo` `→` insertar registros de prueba dentro de la base de datos donde los registros se almacenan en `→ /api/datos`

Logramos que no se pierdan los datos cuando se reinicia el servidor Flask



6. Creo un archivo [simulador.py](#)

```
import sqlite3
import random
import time

# Conexion a SQLite
conexion = sqlite3.connect("sensores.db")
cursor = conexion.cursor()

# Configuracion
cantidad = int(input("Cantidad de capturas: "))
intervalo = int(input("Intervalo entre mediciones (segundos): "))

for i in range(cantidad):

    # Generar valores aleatorios
    co2 = random.randint(400, 1200)
    temperatura = round(random.uniform(18, 32), 2)
    humedad = round(random.uniform(30, 80), 2)

    # Insertar CO2
    cursor.execute("""
INSERT INTO sensores (sensor, valor, unidad)
VALUES (?, ?, ?)
""", ("CO2", co2, "ppm"))

    # Insertar temperatura
    cursor.execute("""
INSERT INTO sensores (sensor, valor, unidad)
VALUES (?, ?, ?)
""", ("Temperatura", temperatura, "°C"))

    # Insertar humedad
    cursor.execute("""
INSERT INTO sensores (sensor, valor, unidad)
VALUES (?, ?, ?)
""", ("Humedad", humedad, "%"))
```

```
conexion.commit()

print(f"\nCaptura {i+1}")
print(f"C02: {co2} ppm")
print(f"Temperatura: {temperatura} °C")
print(f"Humedad: {humedad} %")

time.sleep(intervalo)

conexion.close()

print("\nSimulación finalizada")
```

```
miabertola@Mia:~$ nano simulador.py
miabertola@Mia:~$ python3 simulador.py
Cantidad de capturas: 5
Intervalo entre mediciones (segundos): 2

Captura 1
C02: 1149 ppm
Temperatura: 29.16 °C
Humedad: 31.22 %

Captura 2
C02: 1095 ppm
Temperatura: 24.71 °C
Humedad: 67.78 %

Captura 3
C02: 590 ppm
Temperatura: 31.63 °C
Humedad: 35.48 %

Captura 4
C02: 626 ppm
Temperatura: 19.24 °C
Humedad: 59.44 %

Captura 5
C02: 402 ppm
Temperatura: 20.5 °C
Humedad: 36.9 %

Simulación finalizada
```



```

miabertola@Mia:~$ sqlite3 sensores.db
SQLite version 3.45.1 2024-01-30 16:01:20
Enter ".help" for usage hints.
sqlite> SELECT * FROM sensores;
1|temperatura|24.5|°C|2026-05-09 21:57:54
2|CO2|1149.0|ppm|2026-05-09 22:09:57
3|Temperatura|29.16|°C|2026-05-09 22:09:57
4|Humedad|31.22|%|2026-05-09 22:09:57
5|CO2|1095.0|ppm|2026-05-09 22:10:00
6|Temperatura|24.71|°C|2026-05-09 22:10:00
7|Humedad|67.78|%|2026-05-09 22:10:00
8|CO2|590.0|ppm|2026-05-09 22:10:02
9|Temperatura|31.63|°C|2026-05-09 22:10:02
10|Humedad|35.48|%|2026-05-09 22:10:02
11|CO2|626.0|ppm|2026-05-09 22:10:04
12|Temperatura|19.24|°C|2026-05-09 22:10:04
13|Humedad|59.44|%|2026-05-09 22:10:04
14|CO2|402.0|ppm|2026-05-09 22:10:06
15|Temperatura|20.5|°C|2026-05-09 22:10:06
16|Humedad|36.9|%|2026-05-09 22:10:06

```

7. El sistema combina mediciones simuladas internas (CO₂, temperatura y humedad) con datos climáticos externos obtenidos dinámicamente desde OpenWeatherMap mediante geolocalización IP. Todos los datos son almacenados en SQLite para su posterior análisis. Creo archivo [simulador2.py](#)

```

import sqlite3
import random
import time
import requests
import geocoder

OPENWEATHER_API_KEY = "a9419f344848e87145ad886c36cc6497"

def geo_latlon():
    g = geocoder.ip("me")

    if not g.latlng:
        raise Exception("No se pudo obtener la ubicación")

    lat, lon = g.latlng

    url = (
        "https://api.openweathermap.org/data/2.5/weather?"
        f"lat={lat}&lon={lon}"
        f"&appid={OPENWEATHER_API_KEY}"
        "&units=metric"
        "&lang=es"
    )

    response = requests.get(url, timeout=10)
    data = response.json()

    main = data["main"]
    weather = data["weather"][0]

    return {
        "temp_ext": main["temp"],
        "presion": main["pressure"],
        "humedad_ext": main["humidity"],
        "descripcion": weather["description"]
    }

```



```

    }

conexion = sqlite3.connect("sensores.db")
cursor = conexion.cursor()

cantidad = int(input("Cantidad de capturas: "))
intervalo = int(input("Intervalo entre mediciones: "))

for i in range(cantidad):

    # Sensores internos
    temp_int = round(random.uniform(18, 30), 2)
    humedad_int = round(random.uniform(30, 80), 2)
    co2 = random.randint(400, 1200)

    # Clima externo
    clima = geo_latlon()

    datos = [
        ("CO2", co2, "ppm"),
        ("Temp Interna", temp_int, "°C"),
        ("Humedad Interna", humedad_int, "%"),
        ("Temp Externa", clima["temp_ext"], "°C"),
        ("Presion", clima["presion"], "hPa"),
        ("Humedad Externa", clima["humedad_ext"], "%")
    ]

    for sensor, valor, unidad in datos:

        cursor.execute("""
            INSERT INTO sensores (sensor, valor, unidad)
            VALUES (?, ?, ?)
            """, (sensor, valor, unidad))

    conexion.commit()

    print(f"\nCaptura {i+1}")
    print("CO2:", co2)
    print("Temp interna:", temp_int)
    print("Humedad interna:", humedad_int)
    print("Clima exterior:", clima["descripcion"])

    time.sleep(intervalo)

conexion.close()

```

Salida:

```
(jupyter_env) miabertola@Mia:~$ python simulador2.py
Cantidad de capturas: 3
Intervalo entre mediciones: 2

Captura 1
CO2: 525
Temp interna: 20.49
Humedad interna: 56.69
Clima exterior: muy nuboso

Captura 2
CO2: 613
Temp interna: 23.75
Humedad interna: 43.84
Clima exterior: muy nuboso

Captura 3
CO2: 771
Temp interna: 29.27
Humedad interna: 50.94
Clima exterior: muy nuboso
```

8. La aplicación desarrollada sigue parcialmente una arquitectura Cliente-Servidor REST, ya que existe una separación clara entre el cliente que realiza solicitudes HTTP y el servidor remoto que provee información climática mediante una API web.

Cliente

El cliente es el programa desarrollado en Python (`simulador.py`), ya que:

- realiza solicitudes HTTP utilizando la librería `requests`
- consume recursos externos de una API climática
- procesa la respuesta recibida
- almacena los datos obtenidos en SQLite

La función `geo_latlon()` actúa como consumidor de servicios REST.

Servidor

El servidor es la API de OpenWeatherMap: [OpenWeatherMap](#)

Este servidor:

- recibe solicitudes HTTP GET
- procesa parámetros como latitud y longitud
- responde información climática en formato JSON

Características REST que se cumplen

- Comunicación mediante HTTP

- La interacción se realiza usando solicitudes HTTP (`requests.get()`).
- Uso de recursos identificables mediante URL
 - La API utiliza endpoints claramente definidos: `https://api.openweathermap.org/data/2.5/weather`
- Intercambio de datos en formato JSON
 - La respuesta del servidor se recibe en formato JSON y luego se procesa en Python.
- Arquitectura cliente-servidor desacoplada
 - El cliente no necesita conocer la implementación interna del servidor; solo consume la API.
- Stateless
 - Cada solicitud HTTP contiene toda la información necesaria (latitud, longitud y API key), por lo que el servidor no mantiene estado de sesiones previas.

Elementos REST que faltan o no se implementaron completamente

- **API propia**

El trabajo consume una API REST externa, pero no implementa un servidor REST propio.

- **Métodos HTTP adicionales**

Solo se utiliza el método GET. No se implementan:

- POST
- PUT
- DELETE
- **Recursos propios expuestos**

La aplicación no expone endpoints para que otros clientes consulten los sensores.

- **Cabeceras y códigos HTTP personalizados**

No se gestionan explícitamente headers HTTP ni respuestas REST propias.

Conclusión

El sistema implementa correctamente el rol de cliente REST al consumir datos climáticos externos mediante HTTP y JSON. El servidor REST corresponde a OpenWeatherMap. La arquitectura cumple los principios básicos de REST del lado cliente, aunque no se desarrolló un servidor REST propio completo.

9. Comparación entre el modelo REST actual y el modelo del TP1

El modelo desarrollado en este trabajo presenta diferencias importantes respecto al implementado en el TP1 basado en sockets.

TP1 – Modelo Cliente/Servidor con sockets

En el TP1 se trabajó con una arquitectura cliente-servidor tradicional utilizando sockets TCP o UDP.

Características principales

- El cliente y el servidor fueron desarrollados manualmente.
- La comunicación se realizaba directamente mediante sockets.
- Era necesario definir:
 - IP
 - puerto
 - protocolo
 - formato de mensajes
- El servidor debía permanecer escuchando conexiones.
- El intercambio de datos era de bajo nivel.

Ventajas

- Mayor control sobre la comunicación.
- Más eficiencia y flexibilidad.
- Permite implementar protocolos personalizados.

Desventajas

- Mayor complejidad de programación.
- Necesidad de gestionar conexiones y errores manualmente.
- Menor interoperabilidad.

TP2 – Modelo REST

En este TP se utiliza una arquitectura basada en servicios REST consumidos mediante HTTP.

Características principales

- El cliente utiliza solicitudes HTTP mediante la librería `requests`.
- La comunicación se realiza con una API web externa.
- Los datos se reciben en formato JSON.
- No es necesario administrar sockets manualmente.
- El acceso a recursos se realiza mediante URLs.

Ventajas

- Implementación más simple.
- Mayor interoperabilidad.
- Uso de estándares web ampliamente soportados.
- Fácil integración con APIs externas.

Desventajas

- Menor control sobre la comunicación.
- Dependencia de servicios externos.
- Mayor sobrecarga debido al uso de HTTP.

Conclusión

El TP1 se enfocó en comprender la comunicación cliente-servidor a bajo nivel mediante sockets, mientras que este TP trabaja sobre una capa de mayor abstracción utilizando servicios REST y HTTP. Ambos modelos siguen una arquitectura cliente-servidor, pero REST simplifica la interacción mediante estándares web y APIs reutilizables.

10. Se desarrolló una aplicación cliente-servidor basada en WebSockets utilizando Python, Flask y Flask-SocketIO. El objetivo fue implementar una interfaz web interactiva capaz de establecer comunicación en tiempo real con un servidor. El servidor WebSocket fue implementado en Python mediante Flask-SocketIO y permanece escuchando conexiones de clientes en el puerto 5000. Cuando un cliente se conecta desde el navegador web, se establece un canal de comunicación bidireccional persistente.

```
GNU nano 7.2 server.py *
from flask import Flask, render_template
from flask_socketio import SocketIO, send

app = Flask(__name__)
app.config['SECRET_KEY'] = 'secret!'
socketio = SocketIO(app)

@app.route('/')
def index():
    return render_template('index.html')

@socketio.on('message')
def handle_message(msg):
    print("Mensaje recibido:", msg)

    respuesta = f"Servidor recibió: {msg}"
    send(respuesta, broadcast=True)

if __name__ == '__main__':
    socketio.run(app, host='0.0.0.0', port=5000)
|
```

```

GNU nano 7.2 templates/index.html *
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>Cliente WebSocket</title>

  <script src="https://cdn.socket.io/4.7.2/socket.io.min.js"></script>
</head>

<body>

  <h1>Cliente WebSocket</h1>

  <p id="estado">Desconectado</p>

  <input type="text" id="mensaje" placeholder="Escribí un mensaje">
  <button onclick="enviarMensaje()">Enviar</button>

  <h3>Mensajes:</h3>
  <div id="mensajes"></div>

  <script>
    const socket = io();

    const estado = document.getElementById("estado");
    const mensajes = document.getElementById("mensajes");

    socket.on('connect', () => {
      estado.innerHTML = "Conectado";
    });

    socket.on('disconnect', () => {
      estado.innerHTML = "Desconectado";
    });

    socket.on('message', (msg) => {
      mensajes.innerHTML += `<p>${msg}</p>`;

```

```

      mensajes.innerHTML += `<p>${msg}</p>`;
    });

    function enviarMensaje() {
      const input = document.getElementById("mensaje");

      socket.send(input.value);

      input.value = "";
    }
  </script>

</body>
</html>

```



127.0.0.1:5000



Cliente WebSocket

Conectado

Mensajes:

```
(jupyter_env) miabertola@Mia:~$ nano server.py
(jupyter_env) miabertola@Mia:~$ mkdir templates
(jupyter_env) miabertola@Mia:~$ nano templates/index.html
(jupyter_env) miabertola@Mia:~$ nano templates/index.html
(jupyter_env) miabertola@Mia:~$ python server.py
* Serving Flask app 'server'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.28.233.148:5000
Press CTRL+C to quit
127.0.0.1 - - [09/May/2026 23:34:09] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [09/May/2026 23:34:09] "GET /socket.io/?EIO=4&transport=polling&t=PuF5rfl HTTP/1.1" 200 -
127.0.0.1 - - [09/May/2026 23:34:09] "POST /socket.io/?EIO=4&transport=polling&t=PuF5rg9&sid=n0QARZKSK8ykPdrryAAAA HTTP/1.1" 200 -
127.0.0.1 - - [09/May/2026 23:34:09] "GET /socket.io/?EIO=4&transport=polling&t=PuF5rgC&sid=n0QARZKSK8ykPdrryAAAA HTTP/1.1" 200 -
127.0.0.1 - - [09/May/2026 23:34:09] "GET /favicon.ico HTTP/1.1" 404 -
Mensaje recibido: holaaa
```